TP2 SOAP - Agence de voyage

HAI704I - Architectures logicielles distribuees - 2026-2027. Binome : Mohammed M'BARKI et Mohamed MESTI. Ce document couvre le role B, agence/client de la partie A. Depot : https://github.com/Mbarki0/SOAP-WSDL-TP2

1. Prerequis et architecture

Java 17 minimum, Maven 3.9 minimum et un acces a Maven Central pour la premiere construction. Environnement effectivement verifie : Windows 11, Oracle JDK 23 (build 23+37), Maven 3.9.14, terminal PowerShell 7. Le compilateur utilise release 17. Le programme console utilise uniquement la bibliotheque standard Java pour son interface.

- source/hotel : serveur de A, avec point d'acces SOAP, metier et depot en memoire.

- source/agence : application cliente autonome, sans dependance Maven ni import vers hotel.

- contract : contrat commun en lecture seule, version documentaire contract-v1, recu au commit 63a2f0c.

- captures/agence : echanges reels expurges et transcriptions des essais.

- documentation/agence-notes.md : exigences, decisions, observations et usage de l'IA.

Chemin d'un appel : AgenceConsole -> AgenceService -> HotelGateway -> SoapHotelGateway -> proxy HotelPort -> hotel distant. La console connait seulement les modeles du metier. L'adaptateur SOAP traduit ces modeles vers les classes engendrees et traduit les cinq fautes en RefusMetier. Une panne devient IncidentTechnique.

2. Construire les projets

Ouvrir un terminal a la racine du depot SOAP-WSDL-TP2. Verifier java -version et mvn -v. Sur la machine de verification, si JAVA_HOME n'est pas configure :

```
$env:JAVA_HOME = 'C:\Program Files\Java\jdk-23'
$env:Path = "$env:JAVA_HOME\bin;$env:Path"
```

Adapter ce chemin au JDK installe sur l'autre ordinateur. Construire dans cet ordre :

```
cd source/hotel
mvn clean package
cd ../agence
mvn clean package
```

Le JAR de l'agence est target/agence-soap-1.0-SNAPSHOT.jar. Les JAR et target ne font pas partie du code source a rendre. Dependances principales : com.sun.xml.ws:jaxws-rt:4.0.5 ; tests JUnit Jupiter 5.11.4. Plugins : jaxws-maven-plugin 4.0.5, compiler 3.13.0, surefire 3.5.2 et shade 3.6.0.

3. Comprendre la generation du client

La phase generate-sources execute wsimport depuis ../../contract/hotel.wsdl et son import relatif hotel.xsd. Les 20 fichiers Java engendres sont sous target/generated-sources/wsimport/agence/soap/genere. On y trouve HotelService, HotelPort, RechercheOffres, Offre, DemandeReservation, ConfirmationReservation et les fautes typees. Ils ne sont ni copies dans src/main/java, ni modifies, ni versionnes. mvn clean les supprime ; mvn compile les regenere.

Le build embarque WSDL et XSD dans wsdl/ du JAR. Ainsi, creer le client ne telecharge aucun contrat depuis un hotel actif. L'adresse du WSDL est remplacee a l'execution par BindingProvider.ENDPOINT_ADDRESS_PROPERTY.

4. Demarrer les trois processus

Demarrer d'abord les deux hotels, chacun dans son terminal situe dans source/hotel :

```
java -jar target/hotel-soap-1.0-SNAPSHOT.jar hotel-arenes.properties
```

```
java -jar target/hotel-soap-1.0-SNAPSHOT.jar hotel-port.properties
```

Adresses : Hotel des Arenes sur http://localhost:8081/hotel ; Hotel du Port sur http://localhost:8082/hotel. Les deux hotels sont a Montpellier, avec deux chambres chacun, respectivement 4 et 3 etoiles. Les partenaires fictifs sont AG-ALPHA et AG-BETA. Demander a A leurs secrets fictifs d'origine ; ne pas les mettre dans Git.

Dans un troisieme terminal situe dans source/agence, demarrer avec la saisie masquee du secret :

```
.\scripts\demarrer.ps1
```

Ou, si AGENCE_SECRET est deja defini dans l'environnement du processus :

```
java -jar target/agence-soap-1.0-SNAPSHOT.jar
```

Sans secret, la recherche reste disponible ; la reservation est refusee localement avec une explication. Le secret est conserve en memoire pour la session, jamais journalise. Les tableaux de caracteres sont effaces a la fermeture.

5. Changer la configuration sans recompiler

Copier src/main/resources/agence.properties en agence.local.properties, puis modifier ce fichier externe. Les fichiers *.local.properties sont ignores par Git dans ce projet. La variable AGENCE_SECRET a priorite sur agence.secret.

```
agence.identifiant=AG-BETA
hotels=arenes,port
hotel.arenes.nom=Hotel des Arenes
hotel.arenes.url=http://localhost:8081/hotel
hotel.port.nom=Hotel du Port
hotel.port.url=http://localhost:8082/hotel
delai.connexion.ms=1500
delai.reponse.ms=2500
```

```
java -jar target/agence-soap-1.0-SNAPSHOT.jar agence.local.properties
```

Le fichier est lu en UTF-8. Une adresse invalide est signalee pour l'hotel concerne, sans bloquer les autres. Les delais Metro 4.0.5 utilisent com.sun.xml.ws.connect.timeout et com.sun.xml.ws.request.timeout, en millisecondes entieres. Les recherches sont sequentielles ; leur duree maximale cumule les attentes des hotels configures.

6. Utiliser la console

- Choix 1 : rechercher par ville, dates AAAA-MM-JJ, nombre de personnes, etoiles minimum et budget TOTAL du sejour en EUR.

- Les offres sont triees par devise, prix total croissant puis nom d'hotel. Les hotels du jeu fourni utilisent tous EUR.

- Choix 2 : choisir le numero d'offre, saisir le nom et le prenom du voyageur, puis les donnees FICTIVES de paiement.

- Choix 3 : renvoyer la derniere demande avec le meme identifiant ; saisir de nouveau le paiement, car il n'est pas conserve.

- Choix 4 : revoir les offres ; choix 0 : quitter. Une fin de flux quitte egalement proprement.

Une nouvelle reservation cree un nouvel UUID avant l'appel. Un reessai conserve celui de la demande precedente. Si une reponse est perdue, ne pas creer une nouvelle reservation : utiliser le choix 3. L'hotel detecte

le rejeu et renvoie la meme reference. L'agence controle l'identifiant, les dates, le prix et la devise de la confirmation avant d'annoncer le succes.

Paiement fictif attendu : 16 chiffres pour le numero, mois 1 a 12, expiration non revolue et CVC de 3 chiffres. En vrai terminal Java, numero et CVC sont saisis sans echo. Dans une console d'IDE ou une entree redirigee, l'application ne les reaffiche jamais ; l'IDE peut afficher la frappe. Le paiement n'est ni garde pour le prochain essai, ni ecrit dans un fichier. Les DTO SOAP sont nettoyes apres l'appel. Les Strings imposees par JAXB ne sont pas effacables en place : aucun effacement absolu de la memoire JVM n'est revendique.

7. Tests reproductibles

Depuis source/agence, les tests independants ne demarrent aucun serveur :

```
mvn test
```

Apres construction de source/hotel, les tests d'integration lancent et arretent eux-memes de vraies JVM hotel sur des ports libres :

```
mvn -Dintegration.hotel=true clean package
```

Resultat observe : 25 tests reussis, aucun echec ; dont 4 tests d'integration. Sans le parametre, ces 4 tests sont ignores. Les configurations originales de A sont testees en recherche. Pour les reservations automatisees, les tests creent dans target/hotels des copies de configuration avec des empreintes de secrets fictifs de test. Ils ne changent ni les sources, ni les configurations versionnees de A. Les secrets fictifs d'origine restent necessaires pour reserver avec les fichiers originaux.

Verifier aussi le JAR final, dans un nouveau processus client et avec deux processus hotel :

```
java -Dbasedir=. -cp "target/test-classes;target/agence-soap-1.0-SNAPSHOT.jar"
agence.soap.VerificationJar
```

Sous Linux/macOS, remplacer le separateur ; du classpath par :. Cette commande a ete executee avec succes sous Windows. Elle couvre la CLI reelle, les saisies invalides, une reservation, son rejeu, une offre devenue indisponible et un hotel arrete. Elle controle la sortie avant d'enregistrer des transcriptions sans secret, numero complet ou CVC.

8. Rendu et limites

Les captures sous captures/agence proviennent des vrais echanges SOAP des tests ; les champs sensibles sont expurges AVANT ecriture. Elles montrent SOAP 1.1, les reponses HTTP 200/500, le prix confirme et les fautes typees. Les requetes expurgees ne sont pas directement rejouables. Le programme ne lance aucun paiement reel. Aucun bonus Spring, persistance, image ou interface graphique n'est implemente. date-demo, employe-demo et les deux annexes SOAP n'etaient pas disponibles dans les ressources locales ; le point de controle 0 ne peut pas etre declare valide sur cette base.

Avant le rendu commun : recuperer les secrets fictifs de A, refaire une demonstration humaine, relire tout le code a deux, completer le rapport commun et enregistrer la video demandee par le sujet. Ce README PDF couvre l'agence ; il doit accompagner la documentation de l'hotel. Le rapport et la video communs ne sont pas remplaces par les tests automatises.